Student projects using all database types

Each project is one program that makes a decision and uses five stores: PostgreSQL (relational), MongoDB (document), Redis (key-value), Neo4j (graph), and a vector store. The vector part is built twice, in pgvector and in Qdrant, and the two are compared. Each store holds the data it fits, and the decision uses all of them.

1. Warranty claim service

An electronics shop decides whether a warranty claim is accepted, rejected, or needs more information.

Store	Holds	Required queries
PostgreSQL	customers, orders, order items, products, claims and decisions; FOREIGN KEY	the claim transaction reads the delivered order and rejects a claim dated before delivery; the claim is saved with its decision and evidence in the same transaction
MongoDB	claim submissions, the system of record for them; fields differ by product type (serial number, battery cycles, firmware version); `$jsonSchema` per type	inserting a submission without a field required for its type fails with a validation error
Redis	the decision per claim, key `claim:{id}:decision`, with a TTL	cache-aside read; update an order so the decision changes, show the stale cached value, then delete the key in the same update path so the next read reloads the new decision
Neo4j	product → component → supplier batch	Cypher returns the ids of products that contain a component from a recalled batch; one PostgreSQL query then returns the customers who bought those products
pgvector, Qdrant	warranty policy passages with product category as metadata	the 5 passages most relevant to the claim text, filtered by category

Measured: 30 test claims with the expected decision; recall@5 and MRR of the policy passages on 20 labelled claims.

2. Expense approval

A company decides whether an expense is approved, rejected, or sent to a higher approver.

Store	Holds	Required queries
PostgreSQL	employees, cost centres, budgets with a version column, expenses, decisions	approve in one transaction with `UPDATE budgets ... WHERE id = $id AND version = $version AND remaining >= $amount`; when no row is updated, reject and reload. Two concurrent approvals of the same budget: one commits, one gets a version conflict

Store	Holds	Required queries
MongoDB	receipts, the system of record for them; fields differ by vendor and country; `$jsonSchema` with `validationAction: "error"`	inserting a receipt without the required VAT number or total fails with a validation error
Redis	the approver chain per employee, with a TTL	show the cached chain still allowing an approval after a delegation was revoked; then delete the key when a delegation changes
Neo4j	`(:Employee)-[:DELEGATED_TO {scope, amount_limit, valid_from, valid_to}]->(:Employee)`, `(:Employee)-[:RESPONSIBLE_FOR]->(:CostCentre)`	for an expense's cost centre, amount and date: is there a path to the candidate approver whose every delegation covers that scope, amount and date?
pgvector, Qdrant	receipt texts and expense policy passages, with a `document_type` field	near-duplicate receipts submitted twice; the policy passages relevant to an expense

Measured: 25 test expenses with the expected outcome; precision and recall of duplicate detection on 25 labelled pairs; recall@5 and MRR of policy passages on 20 labelled expenses.

3. Course enrolment advisor

A university decides whether a student may enrol in a course, and suggests courses that match the student's interest.

Store	Holds	Required queries
PostgreSQL	students, courses, sections with `seats_taken` and `capacity`, enrolments, grades; UNIQUE (student_id, section_id); CHECK (seats_taken <= capacity)	enrol in one transaction: lock the section row with `SELECT ... FOR UPDATE`, check `seats_taken < capacity`, insert the enrolment, increment `seats_taken`. Two students taking the last seat: one enrolment commits, the other is rejected
MongoDB	course descriptions, the system of record for them; structure differs by course (labs, projects, reading lists)	courses that have both a lab and an exam
Redis	the student's planned courses (session), with a TTL	the plan survives a page reload and expires after inactivity
Neo4j	prerequisites, equivalent courses, and `(:Student)-[:COMPLETED]->(:Course)` copied from recorded grades	courses whose prerequisites the student has completed; the shortest prerequisite path to a target course
pgvector, Qdrant	course descriptions with semester and credits as metadata	courses similar to a free-text interest, filtered by semester

Measured: 15 eligibility test cases; recall@5 and MRR on 20 labelled interest queries.

4. IT incident triage

An IT department decides, for a new ticket, which services are affected, which team gets it, and which known fix to try first.

Store	Holds	Required queries
PostgreSQL	tickets, assets, services, teams, `service_team(service_id, team_id)`, service-level targets; FOREIGN KEY from ticket to asset	route a new ticket to the team that owns its affected service; open tickets past their target time, per team
MongoDB	raw monitoring alerts, the system of record for them; fields differ by source (server, network, application)	with an `{asset_id: 1, observed_at: 1}` index, all alerts for one asset in a time range
Redis	a heartbeat key per service, set with a TTL by the service	a service whose key has expired is reported as down
Neo4j	dependencies: application → service → server → network device	all applications affected when a server is down; common dependencies of two failing applications
pgvector, Qdrant	past incidents with their resolution and knowledge-base articles, with service as metadata	the 5 most similar past incidents or articles for the new ticket, filtered by affected service

Measured: 20 tickets labelled with the correct article or past incident, recall@5 and MRR; 10 impact test cases; 10 routing test cases.

5. Recipe and grocery service with allergens

A grocery service suggests recipes that are safe for the customer’s allergies and whose ingredients are in stock.

Store	Holds	Required queries
PostgreSQL	products, stock, prices, customers with their allergy codes, orders	can every product of a recipe be bought now, and at what total price?
MongoDB	recipes, the system of record for them, with the product ids they use; steps and nutrition fields differ by recipe	recipes under 30 minutes that use a given product
Redis	the shopping cart per customer, with a TTL	add, remove, read the cart; the cart expires after inactivity
Neo4j	product → ingredient → allergen; ingredient substitutes	with the recipe’s product ids and the customer’s allergy codes as parameters: every allergen reachable from those products, and substitutes that contain none of those codes
pgvector, Qdrant	recipe descriptions with `cuisine`, `preparation_minutes` and `allergen_codes` as metadata	recipes similar to a free-text request (“quick vegetarian dinner”); the SQL `WHERE` clause and the Qdrant filter both exclude every code in the customer’s allergy list

Measured: 20 cases of an allergy list and a request, at least 5 with several allergies; the number of unsafe recipes returned, which must be 0; recall@5 on 20 labelled requests.

Requirements for every project

- **Setup:** one `docker compose up` starts PostgreSQL with pgvector (`pgvector/pgvector:pg16`), MongoDB, Redis, Neo4j and the Qdrant server (`qdrant/qdrant`). Library versions are pinned.
- **Data:** realistic, 200–1,000 records per store, generated or from a public dataset, with its source noted. PostgreSQL is the system of record for the business entities and decisions; MongoDB is the system of record for the flexible source documents named in the project. Redis, graph and vector records are derived; they keep the source id and the time they were refreshed.
- **One reason per store:** for each store, one sentence on why this data is kept there.
- **pgvector and Qdrant:** the same normalised embeddings (`all-MiniLM-L6-v2`), cosine distance, text ids and 20 fixed queries in both. For each system, compare its HNSW top-10 with its own exact top-10 and report ANN recall@10 (shared ids / 10). Report the number of shared ids in the two systems' top-10 sets. With a filter that keeps about 5% of the rows, report the number of results returned, ANN recall@10 and latency. Record `hnsw.ef_search` and whether an iterative scan is on in pgvector; create payload indexes on the filtered fields and record `hnsw_ef` in Qdrant. Show one query that pgvector answers in a single SQL statement joined with business tables, and the same query with Qdrant.
- **One update across stores:** change one source record and show what happens to its derived copies (Redis entry, vector, graph node) until they are refreshed.
- **Tests:** the test cases and labels are written before tuning and handed in. Every number in the report is printed by the submitted code.
- **Hand in:** code, data, test cases, and a 3–4 page report: the role of each store, the main queries, the measured results, and one failure case explained.

At Project presentation I (week 7) a team shows the data model of all five stores, the vector search in both pgvector and Qdrant, and the measurement plan.